03-01 TPU (Transaction Processing Unit) 아키텍처

개요

TPU(Transaction Processing Unit)는 Solana 검증자에서 새로운 트랜잭션을 처리하고 블록을 생성하는 핵심 컴포넌트입니다. TPU는 다단계 파이프라인 구조로 설계되어 있으며, 각 단계가 병렬로 동작하여 높은 처리량을 달성합니다.

TPU 구조체 정의

```
pub  struct Tpu {

fetch_stage: FetchStage,

sigverify_stage: SigVerifyStage,

vote_sigverify_stage: SigVerifyStage,

banking_stage: BankingStage,

cluster_info_vote_listener: ClusterInfoVoteListener,

broadcast_stage: BroadcastStage,

tpu_quic_t: thread::JoinHandle<()>,

tpu_forwards_quic_t: thread::JoinHandle<()>,

tpu_entry_notifier: Option<TpuEntryNotifier>,

staked_nodes_updater_service: StakedNodesUpdaterService,

tracer_thread_hdl: TracerThread,
```

```
}
```

TPU는 다음과 같은 주요 컴포넌트들로 구성됩니다:

- **FetchStage**: 네트워크에서 트랜잭션 패킷 수신
- **SigVerifyStage**: 일반 트랜잭션의 서명 검증
- **VoteSigVerifyStage**: 투표 트랜잭션의 서명 검증
- **BankingStage**: 트랜잭션 실행 및 블록 생성
- **BroadcastStage**: 생성된 블록을 네트워크에 전파

TPU 소켓 구성

```
pub struct TpuSockets {
pub transactions: Vec<UdpSocket>,
pub transaction_forwards: Vec<UdpSocket>,
pub vote: Vec<UdpSocket>,
pub broadcast: Vec<UdpSocket>,
pub transactions_quic: UdpSocket,
pub transactions_forwards_quic: UdpSocket,
}
```

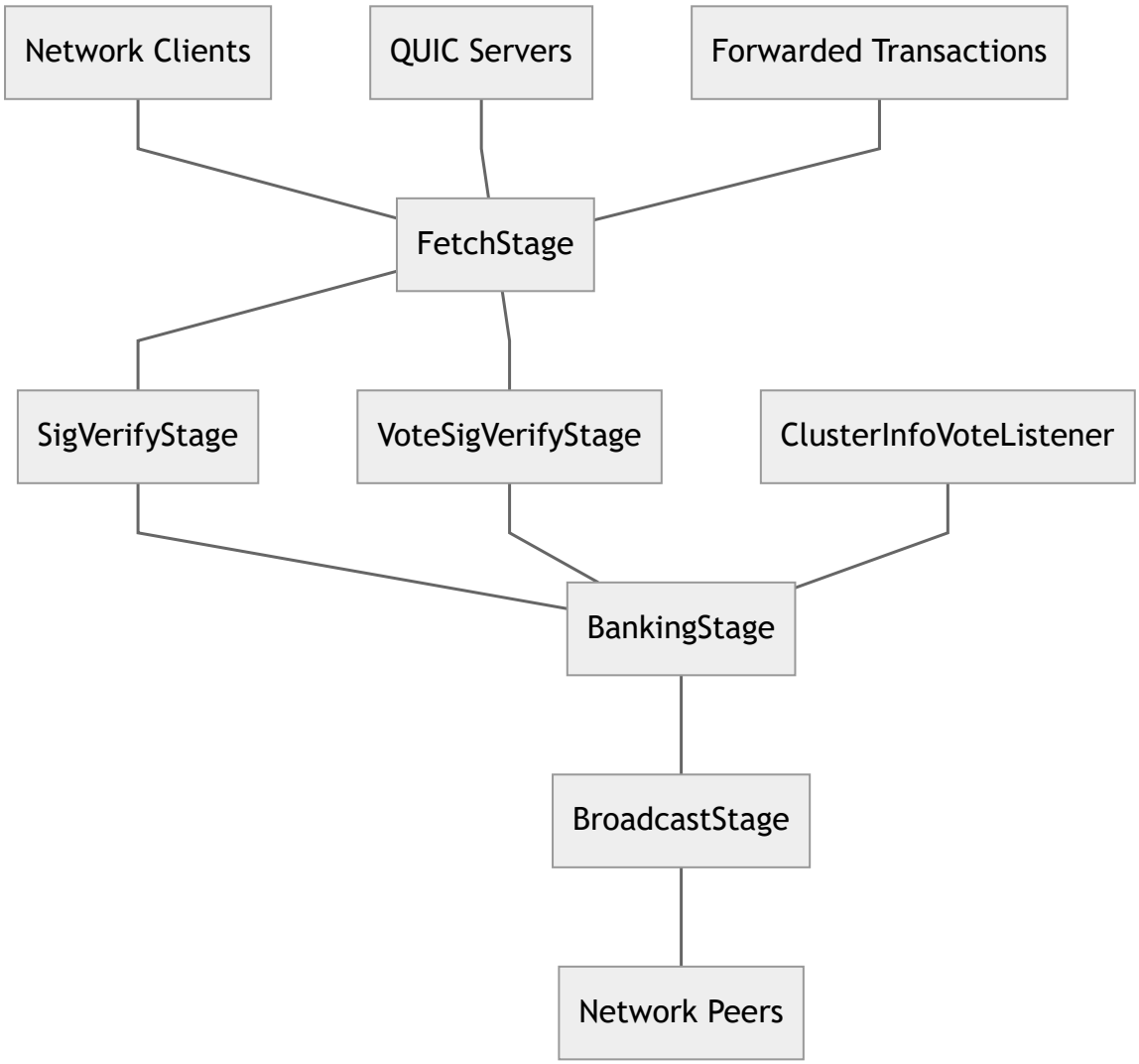
TPU는 다양한 유형의 네트워크 통신을 위해 여러 소켓을 사용합니다:

- **transactions**: 일반 트랜잭션 수신용 UDP 소켓들
- **transaction_forwards**: 전달된 트랜잭션 수신용 UDP 소켓들

- **vote**: 투표 트랜잭션 수신용 UDP 소켓들
- **broadcast**: 블록 전파용 UDP 소켓들
- **transactions_quic**: QUIC 프로토콜 기반 트랜잭션 수신용 소켓
- **transactions_forwards_quic**: QUIC 프로토콜 기반 전달 트랜잭션 수신용 소켓

TPU 파이프라인 구조

1. 데이터 플로우 다이어그램



2. 채널 기반 통신

TPU의 각 단계는 crossbeam 채널을 통해 연결됩니다:

```
// 패킷 수신 채널들
let (packet_sender, packet_receiver) = unbounded();
let (vote_packet_sender, vote_packet_receiver) = unbounded();
let (forwarded_packet_sender, forwarded_packet_receiver) = unbounded();


// 서명 검증된 트랜잭션 채널들
let (non_vote_sender, non_vote_receiver) = banking_tracer.create_channel_non_vot
let (tpu_vote_sender, tpu_vote_receiver) = banking_tracer.create_channel_tpu_vot
let (gossip_vote_sender, gossip_vote_receiver) = banking_tracer.create_channel_g
```

TPU::new() 함수 분석

1. 함수 시그니처

```
pub fn new(
cluster_info: &Arc<ClusterInfo>,
poh_recorder: &Arc<RwLock<PohRecorder>>,
entry_receiver: Receiver<WorkingBankEntry>,
retransmit_slots_receiver: Receiver<Slot>,
sockets: TpuSockets,
subscriptions: &Arc<RpcSubscriptions>,
transaction_status_sender: Option<TransactionStatusSender>,
```

```
entry_notification_sender: Option<EntryNotifierSender>,

blockstore: Arc<Blockstore>,

broadcast_type: &BroadcastStageType,

exit: Arc<AtomicBool>,

shred_version: u16,

vote_tracker: Arc<VoteTracker>,

bank_forks: Arc<RwLock<BankForks>>,

verified_vote_sender: VerifiedVoteSender,

gossip_verified_vote_hash_sender: GossipVerifiedVoteHashSender,

replay_vote_receiver: ReplayVoteReceiver,

replay_vote_sender: ReplayVoteSender,

bank_notification_sender: Option<BankNotificationSender>,

tpu_coalesce: Duration,

duplicate_confirmed_slot_sender: DuplicateConfirmedSlotsSender,

connection_cache: &Arc<ConnectionCache>,

turbine_quic_endpoint_sender: AsyncSender<(SocketAddr, Bytes)>,

keypair: &Keypair,

log_messages_bytes_limit: Option<usize>,

staked_nodes: &Arc<RwLock<StakedNodes>>,

shared_staked_nodes_overrides: Arc<RwLock<HashMap<Pubkey, u64>>>,

banking_tracer: Arc<BankingTracer>,

tracer_thread_hdl: TracerThread,

tpu_enable_udp: bool,
```

```
prioritization_fee_cache: &Arc<PrioritizationFeeCache>,
block_production_method: BlockProductionMethod,
_generator_config: Option<GeneratorConfig>,
) -> (Self, Vec<Arc<dyn NotifyKeyUpdate + Sync + Send>>)
```

2. 초기화 순서

TPU 초기화는 다음 순서로 진행됩니다:

2.1 FetchStage 초기화

```
let  fetch_stage = FetchStage::new_with_sender(
transactions_sockets,
tpu_forwards_sockets,
tpu_vote_sockets,
exit.clone(),
&packet_sender,
&vote_packet_sender,
&forwarded_packet_sender,
forwarded_packet_receiver,
poh_recorder,
tpu_coalesce,
Some(bank_forks.read().unwrap().get_vote_only_mode_signal()),
tpu_enable_udp,
```

```
);
```

FetchStage는 다양한 소켓에서 패킷을 수신하고 적절한 채널로 전달합니다.

2.2 QUIC 서버 초기화

```
let SpawnServerResult {
endpoint: _,
thread: tpu_quic_t,
key_updater,
} = spawn_server(
"solQuicTpu",
"quic_streamer_tpu",
transactions_quic_sockets,
keypair,
packet_sender,
exit.clone(),
MAX_QUIC_CONNECTIONS_PER_PEER,
staked_nodes.clone(),
MAX_STAKED_CONNECTIONS,
MAX_UNSTAKED_CONNECTIONS,
DEFAULT_WAIT_FOR_CHUNK_TIMEOUT,
tpu_coalesce,
).unwrap();
```

QUIC 서버는 고성능 UDP 기반 프로토콜을 사용하여 트랜잭션을 수신합니다.

2.3 SigVerifyStage 초기화

```
let sigverify_stage = {
let verifier = TransactionSigVerifier::new(non_vote_sender);
SigVerifyStage::new(packet_receiver, verifier, "solSigVerTpu", "tpu-verifier")
};

let vote_sigverify_stage = {
let verifier = TransactionSigVerifier::new_reject_non_vote(tpu_vote_sender);
SigVerifyStage::new(
vote_packet_receiver,
verifier,
"solSigVerTpuVot",
"tpu-vote-verifier",
)
};
```

일반 트랜잭션과 투표 트랜잭션을 위한 별도의 서명 검증 단계를 생성합니다.

2.4 ClusterInfoVoteListener 초기화

```
let cluster_info_vote_listener = ClusterInfoVoteListener::new(
exit.clone(),
cluster_info.clone(),
```

```
gossip_vote_sender,
poh_recorder.clone(),
vote_tracker,
bank_forks.clone(),
subscriptions.clone(),
verified_vote_sender,
gossip_verified_vote_hash_sender,
replay_vote_receiver,
blockstore.clone(),
bank_notification_sender,
duplicate_confirmed_slot_sender,
);
```

Gossip 네트워크에서 투표 정보를 수신하고 처리합니다.

2.5 BankingStage 초기화

```
let  banking_stage = BankingStage::new(
block_production_method,
cluster_info,
poh_recorder,
non_vote_receiver,
tpu_vote_receiver,
gossip_vote_receiver,
```

```
transaction_status_sender,
replay_vote_sender,
log_messages_bytes_limit,
connection_cache.clone(),
bank_forks.clone(),
prioritization_fee_cache,
);
```

검증된 트랜잭션들을 실행하고 블록을 생성합니다.

2.6 BroadcastStage 초기화

```
let  broadcast_stage = broadcast_type.new_broadcast_stage(
broadcast_sockets,
cluster_info.clone(),
entry_receiver,
retransmit_slots_receiver,
exit,
blockstore,
bank_forks,
shred_version,
turbine_quic_endpoint_sender,
);
```

생성된 블록을 네트워크의 다른 검증자들에게 전파합니다.

성능 최적화 특징

1. 병렬 처리

- 각 단계가 독립적인 스레드에서 실행
- 파이프라인 구조로 동시 처리 가능

2. 채널 기반 통신

- Lock-free 채널을 사용한 효율적인 데이터 전달
- 백프레셔(backpressure) 제어

3. QUIC 프로토콜 지원

- UDP 기반의 고성능 프로토콜
- 연결 다중화 및 흐름 제어

4. 스테이킹 기반 우선순위

- 스테이킹된 노드에 대한 우선적 처리
- DoS 공격 방어

요약

TPU는 Solana의 트랜잭션 처리 성능을 결정하는 핵심 컴포넌트입니다. 다단계 파이프라인 구조와 병렬 처리를 통해 높은 처리량을 달성하며, QUIC 프로토콜과 스테이킹 기반 우선순위 시스템을 통해 네트워크 효율성과 보안성을 확보합니다.